XeroIMS: Advanced Inventory Management System

Project Report

A report submitted for the course Project - III

Submitted By
Student Name
Semester - VII
Roll No. XXXXXXXXX

Supervised By
Dr. Supervisor Name

Institute
Logo

Date
March 2026

Department of Computer Science and Engineering
Indian Institute of Information Technology, Senapati, Manipur

Declaration

I hereby declare that this report entitled **XeroIMS: Advanced Inventory Management System** is an original work carried out for the course requirements of Project - III. All external references have been duly acknowledged.

Student Name
Roll No. XXXXXXXXX

Date: _____

Institute
Logo

Department of Computer Science and Engineering
Indian Institute of Information Technology, Senapati, Manipur

---

Certificate

This is to certify that the report entitled **XeroIMS: Advanced Inventory Management System** submitted in partial fulfillment of the requirements for Project - III is a bona fide work carried out by **Student Name** (Roll No. **XXXXXXXXX**) under my supervision.

Signature of Supervisor
Dr. Supervisor Name

Abstract

XeroIMS is a full-stack inventory management platform built with Java 21 and Spring Boot 3.5, designed for secure self-hosted business operations. The system combines inventory modules (category, product, supplier, pricing) with transactional workflows (sales, purchase, approval-driven billing, and fund management). It supports dual authentication paths: OAuth 2.0 with Google and OTP-based email verification for account registration and password reset.

The platform includes secure deployment architecture using Docker, Nginx reverse proxy, and TLS-ready production configuration. It also provides enterprise utility features such as PDF invoice/purchase-order generation, Gmail API integration for transactional emails, Excel exports, and dashboard analytics. This report documents the project lifecycle in a format aligned with academic project report expectations: motivation, literature survey, requirements engineering, architecture, implementation details, observed results, limitations, and roadmap.

Acknowledgement

I express sincere gratitude to my supervisor and faculty members for their guidance and support during the development and documentation of this project. I also thank my peers and well-wishers for their valuable suggestions and encouragement.

Contents

Acknowledgement 1

List of Figures 4

List of Tables 5

1 Introduction 6
- 1.1 Background . . . 6
- 1.2 Problem Statement . . . 6
- 1.3 Project Objectives . . . 6
- 1.4 Key Features Implemented . . . 7
- 1.5 Technology Stack . . . 7
- 1.6 Report Organization . . . 7

2 Literature Survey 8
- 2.1 Survey Scope . . . 8
- 2.2 Inventory and ERP-Oriented Systems . . . 8
- 2.3 Spring Boot Architectural Practices . . . 8
- 2.4 Authentication Patterns . . . 8
- 2.5 Account Protection and Abuse Control . . . 8
- 2.6 Document Generation and Transactional Communication . . . 9
- 2.7 Gap Addressed by XeroIMS . . . 9

3 Requirement Engineering 10
- 3.1 Methodology . . . 10
- 3.2 Stakeholders . . . 10
- 3.3 Functional Requirements . . . 10
  - 3.3.1 User and Access Management . . . 10
  - 3.3.2 Inventory Core . . . 10
  - 3.3.3 Transaction and Billing . . . 11
  - 3.3.4 Document and Email . . . 11
  - 3.3.5 Analytics and Reporting . . . 11
- 3.4 Non-Functional Requirements . . . 11
- 3.5 Assumptions and Constraints . . . 11
- 3.6 Use Case Diagram . . . 12

4 System Design 13
- 4.1 High-Level Architecture . . . 13
- 4.2 Context Data Flow (DFD Level-0) . . . 14
- 4.3 Core Domain Model . . . 14
- 4.4 Authentication and Transaction Flow . . . 15
- 4.5 Data Flow Diagrams (DFD) . . . 15

4.5.1 DFD Level 1 . . . 15
4.5.2 DFD Level 2 . . . 16
4.6 Design Decisions . . . 17
4.6.1 Ownership Pattern . . . 17
4.6.2 Approval-Driven Inventory and Finance . . . 17
4.6.3 Billing Snapshot Strategy . . . 17
4.6.4 Security Boundary Placement . . . 17
4.7 Interface Design Notes . . . 17

5 Implementation **18**
5.1 Implementation Overview . . . 18
5.2 Package-Wise Implementation . . . 18
5.2.1 Controllers . . . 18
5.2.2 Services . . . 18
5.2.3 Data Access Layer . . . 18
5.3 Security Implementation . . . 19
5.3.1 Spring Security Configuration . . . 19
5.3.2 Account Lockout . . . 19
5.4 PDF and Email Implementation . . . 19
5.4.1 PDF Pipeline . . . 19
5.4.2 Gmail API Pipeline . . . 19
5.5 Deployment Implementation . . . 19
5.5.1 Container Topology . . . 19
5.5.2 Configuration Management . . . 20
5.6 Implementation Traceability Matrix . . . 20

6 Results and Evaluation **21**
6.1 Validation Approach . . . 21
6.2 Observed Functional Outcomes . . . 21
6.3 Security and Reliability Observations . . . 22
6.4 Performance and Scalability Notes . . . 22
6.5 Known Gaps . . . 22
6.6 Outcome Summary . . . 22
6.7 Result Screenshots . . . 23
6.7.1 Authentication and Account Screens . . . 23
6.7.2 Dashboard Screens . . . 24
6.7.3 Core Module Screens . . . 25
6.7.4 Transaction and Billing Screens . . . 27
6.7.5 Profile Screens . . . 30
6.8 Generated PDF Documents and Delivery Mode . . . 31

7 Conclusion and Future Scope **35**
7.1 Conclusion . . . 35
7.2 Contributions of XeroIMS . . . 35
7.3 Future Scope . . . 35
7.4 Final Remark . . . 35

Bibliography **36**

List of Figures

3.1 Use case coverage for user operations in XeroIMS 12

4.1 Deployment and runtime architecture of XeroIMS 13
4.2 DFD Level-0 for XeroIMS 14
4.3 Core ER model extracted from implemented entities 14
4.4 Combined authentication and billing workflow 15
4.5 DFD Level-1 for XeroIMS 16
4.6 DFD Level-2 for XeroIMS 16

6.1 Login page 23
6.2 Registration page 23
6.3 Dashboard view I 24
6.4 Dashboard view II 24
6.5 Dashboard view III 25
6.6 Category management module 25
6.7 Product management module 26
6.8 Supplier management module 26
6.9 Pricing management module 27
6.10 Sales module 27
6.11 Purchase module 28
6.12 Billing module view I 28
6.13 Billing module view II 29
6.14 Billing module view III 29
6.15 Profile page view I 30
6.16 Profile page view II 30
6.17 Profile page view III 31
6.18 Generated receipt (physical copy with product) 32
6.19 Generated sale-order PDF (emailed) 33
6.20 Generated purchase-order PDF (emailed) 34

List of Tables

1.1 Primary technology choices in XeroIMS 7

3.1 Non-functional requirement summary 11

5.1 Requirement-to-module traceability 20

6.1 Feature validation summary from implementation artifacts 21

Chapter 1

Introduction

1.1 Background

Modern small and medium businesses often manage inventory using spreadsheets or disconnected software, which creates issues in stock accuracy, transaction validation, billing transparency, and security. XeroIMS addresses this by providing an integrated inventory and transaction management platform with secure authentication, financial tracking, and document automation.

1.2 Problem Statement

The target problem is to build a self-hosted, secure, and practical inventory management solution that can:

- Maintain item master data and supplier-based pricing,
- Capture sales and purchase operations with approval controls,
- Keep running financial state (balance, expenses, profit),
- Provide email + PDF artifacts for business communications,
- Offer production-ready deployment hardening.

1.3 Project Objectives

XeroIMS objectives are as follows:

1. Build modular CRUD pipelines for category, product, supplier, and pricing.
2. Support two user onboarding paths: Google OAuth 2.0 and OTP-based registration.
3. Implement pending-to-completed transaction approvals with business rules.
4. Generate PDF invoices, receipts, and purchase orders from templates.
5. Integrate Gmail API for sending HTML transactional emails.
6. Provide dashboard metrics and exportable billing data.
7. Enable Dockerized deployment with reverse proxy security controls.

1.4 Key Features Implemented

- **Authentication**: Spring Security form login, Google OAuth login, OTP verification, and password reset.
- **Security Controls**: login-attempt tracking with lockout, Nginx rate limits, and secure deployment guidance.
- **Inventory Core**: Category, Product, Supplier, Pricing modules with ownership-aware data isolation.
- **Transactions**: Sales and Purchase entries with status transitions (PENDING/COMPLETED/CANCEL
- **Finance**: Billing snapshots, balance updates, expenses/profit tracking, fund transaction log.
- **Documents**: Thymeleaf-to-PDF conversion using Flying Saucer.
- **Communication**: Gmail OAuth authorization and API-based email dispatch.
- **Analytics**: Dashboard charts for inventory, sales, purchases, top products, and category distribution.

1.5 Technology Stack

Table 1.1: Primary technology choices in XeroIMS

Layer	Technology
Backend	Java 21, Spring Boot 3.5, Spring MVC, Spring Security, Spring Data JPA
Database	MySQL 8.0
Frontend	Thymeleaf, HTML/CSS, Bootstrap, static JS/CSS
Auth	OAuth 2.0 client, form login, OTP verification
Email	Gmail API + JavaMail + Spring Mail
PDF	Flying Saucer PDF renderer with Thymeleaf templates
Deployment	Docker, Docker Compose, Nginx reverse proxy, Certbot-ready setup
Build Tool	Maven

1.6 Report Organization

The rest of this report is organized as: literature survey, requirement engineering, architecture and system design, implementation details, results/observations, and concluding roadmap.

Chapter 2

Literature Survey

2.1 Survey Scope

The survey focuses on three domains relevant to XeroIMS: (1) inventory software patterns, (2) secure web application architecture, and (3) document/email automation in business systems.

2.2 Inventory and ERP-Oriented Systems

Open-source inventory and ERP tools (e.g., Odoo modules, ERPNext inventory components) demonstrate common design principles: normalized master data, transactional states, and role-aware operations. XeroIMS adopts a focused subset of this philosophy for SMEs by keeping module boundaries clear and the UI workflow straightforward.

2.3 Spring Boot Architectural Practices

Layered architecture with *controller-service-repository* remains a practical approach for maintainability in enterprise Java systems [1]. In XeroIMS, each business area maps to dedicated service classes, with repositories for ownership-aware queries (user vs oauth user records).

2.4 Authentication Patterns

Hybrid authentication is common in modern platforms: social login for convenience and direct credential-based login for fallback/control [2]. XeroIMS follows this hybrid strategy using:

- Google OAuth 2.0 for one-click sign-in,
- Email/password registration with OTP verification,
- OTP-based password reset for non-OAuth accounts.

2.5 Account Protection and Abuse Control

Credential abuse mitigation combines app-layer and edge-layer controls:

- App-level lockout via failed login persistence in database,
- Reverse proxy rate limiting using Nginx,
- Session/cookie and endpoint-access restrictions in Spring Security.

This defense-in-depth model aligns with practical secure deployment patterns.

2.6 Document Generation and Transactional Communication

Business systems typically generate invoices and purchase documents programmatically. HTML template to PDF conversion is a widely adopted approach because template reuse improves maintainability [3]. XeroIMS uses Thymeleaf templates rendered through Flying Saucer for consistent branded outputs.

For communication, Gmail API is integrated after per-user OAuth authorization. This allows account-specific outbound email while retaining clear user consent.

2.7 Gap Addressed by XeroIMS

Compared to generic CRUD inventory projects, XeroIMS fills the following practical gaps:

- approval-gated inventory and billing transitions,
- combined OTP + OAuth identity management,
- native PDF + email workflow inside transaction modules,
- containerized deployment with Nginx and TLS-oriented design.

Chapter 3

Requirement Engineering

3.1 Methodology

Requirements were derived from implemented controllers, services, entities, repository signatures, and deployment configuration files available in the project.

3.2 Stakeholders

- **Inventory Operator**: manages catalog, transactions, billing, and reporting.
- **Supplier**: receives purchase-order communication and provides inventory.
- **Customer**: receives invoice/receipt and completes sales cycle.
- **System Maintainer**: deploys and secures infrastructure.

3.3 Functional Requirements

3.3.1 User and Access Management

- FR-01: User shall register with name, email, and password.
- FR-02: System shall send 6-digit OTP for verification with expiry.
- FR-03: User shall login via Google OAuth 2.0.
- FR-04: System shall provide forgot-password flow via OTP.
- FR-05: System shall lock account after excessive failed login attempts.

3.3.2 Inventory Core

- FR-06: User shall create/update/delete categories.
- FR-07: User shall create/update/delete products with SKU, price, and quantity.
- FR-08: User shall create/update/delete suppliers with category mapping.
- FR-09: User shall manage supplier-specific pricing entries.

3.3.3 Transaction and Billing

- FR-10: User shall create purchase and sale transactions in PENDING state.
- FR-11: System shall approve or cancel pending transactions.
- FR-12: On purchase approval, stock increases and balance decreases.
- FR-13: On sale approval, stock decreases and balance/profit increase.
- FR-14: System shall reject purchase approval if funds are insufficient.
- FR-15: System shall reject sale approval if stock is insufficient.
- FR-16: System shall maintain billing snapshots and fund transactions.

3.3.4 Document and Email

- FR-17: System shall generate invoice/receipt PDF for sales.
- FR-18: System shall generate purchase-order PDF for purchases.
- FR-19: System shall send HTML order emails through Gmail API.

3.3.5 Analytics and Reporting

- FR-20: Dashboard shall show summary metrics and trend charts.
- FR-21: Billing module shall export filtered sales and purchases to Excel.

3.4 Non-Functional Requirements

Table 3.1: Non-functional requirement summary

Category	**Requirement**
Security	OAuth2 support, password hashing, login lockout, edge rate limiting, DB not publicly exposed in default compose setup.
Reliability	Health checks for app/mysql/nginx containers, persistent Docker volumes, transaction-scoped updates for critical status transitions.
Maintainability	Layered package structure, DTO usage, centralized service logic, exception handling strategy.
Scalability	Reverse proxy front-end and containerized deployment allow horizontal evolution and cloud migration.
Usability	Form-driven workflows for category/product/supplier/pricing/sales/purchase/billing and clear feedback messages.
Portability	One-command local startup via Docker Compose; Maven build for standalone app runs.

3.5 Assumptions and Constraints

- Single-admin-style trust model per account (full module access).

- One database engine target: MySQL 8.0.
- Gmail send functionality requires user authorization and valid Google credentials.
- Production TLS requires external domain and certificate setup.

3.6 Use Case Diagram

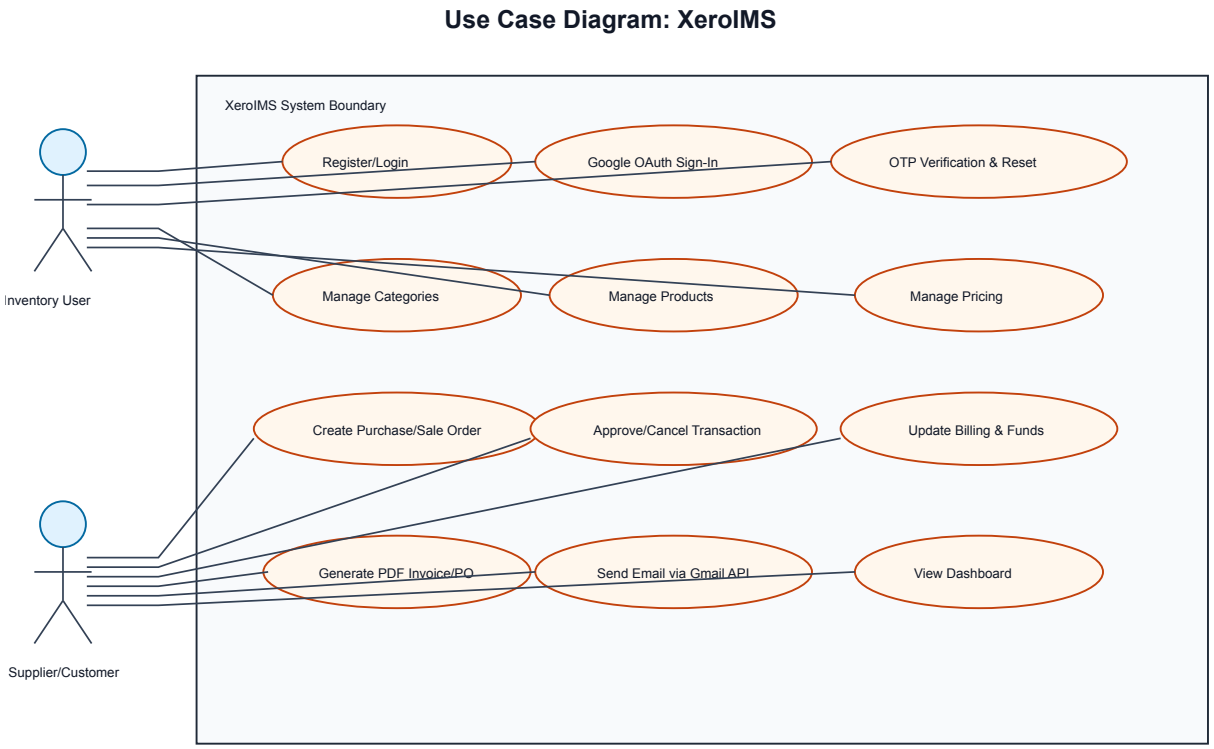


Figure 3.1: Use case coverage for user operations in XeroIMS

Chapter 4

System Design

4.1 High-Level Architecture

XeroIMS follows a layered architecture with reverse proxy edge, Spring Boot application core, and MySQL persistence layer.

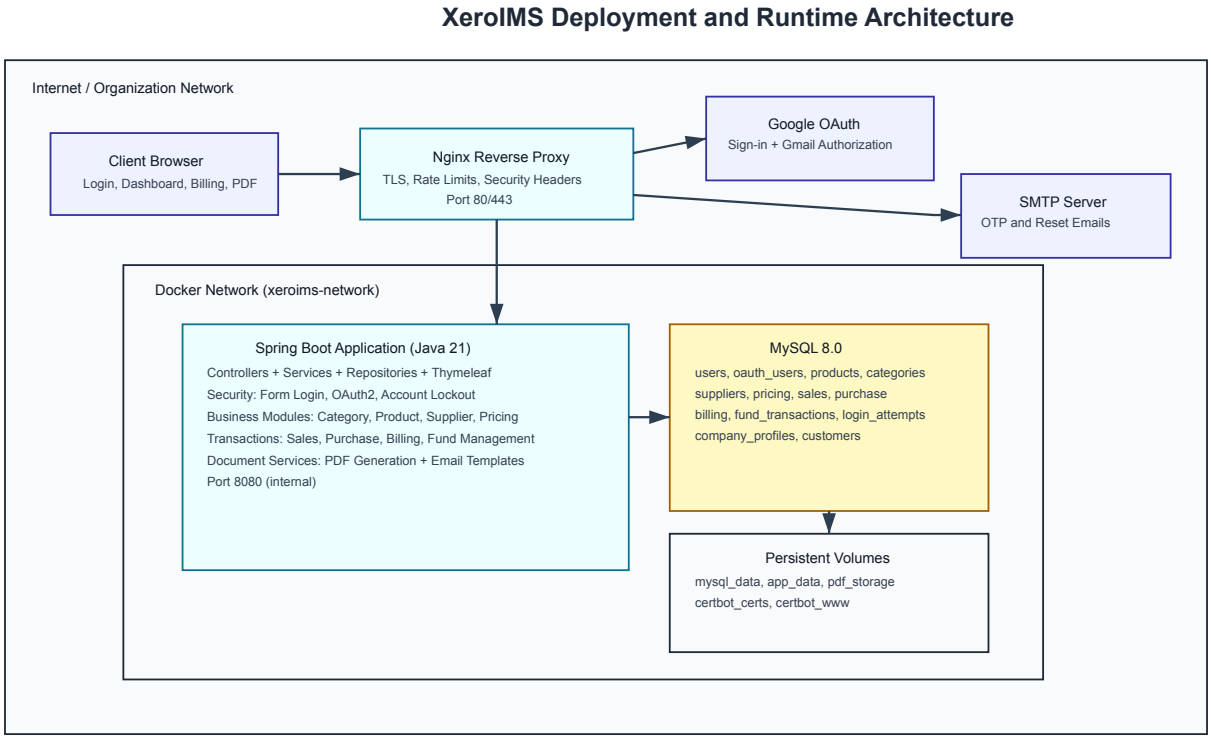


Figure 4.1: Deployment and runtime architecture of XeroIMS

4.2 Context Data Flow (DFD Level-0)

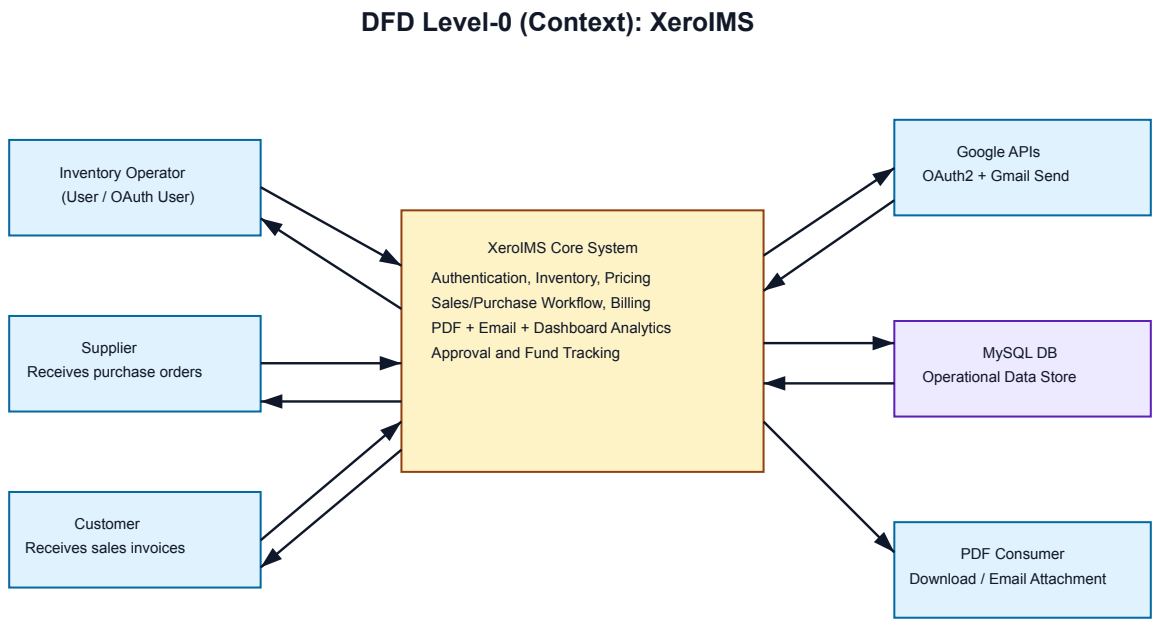


Figure 4.2: DFD Level-0 for XeroIMS

4.3 Core Domain Model

The data model supports ownership-aware records via either `user_id` or `oauth_user_id`. This allows both traditional and OAuth users to use equivalent business modules.

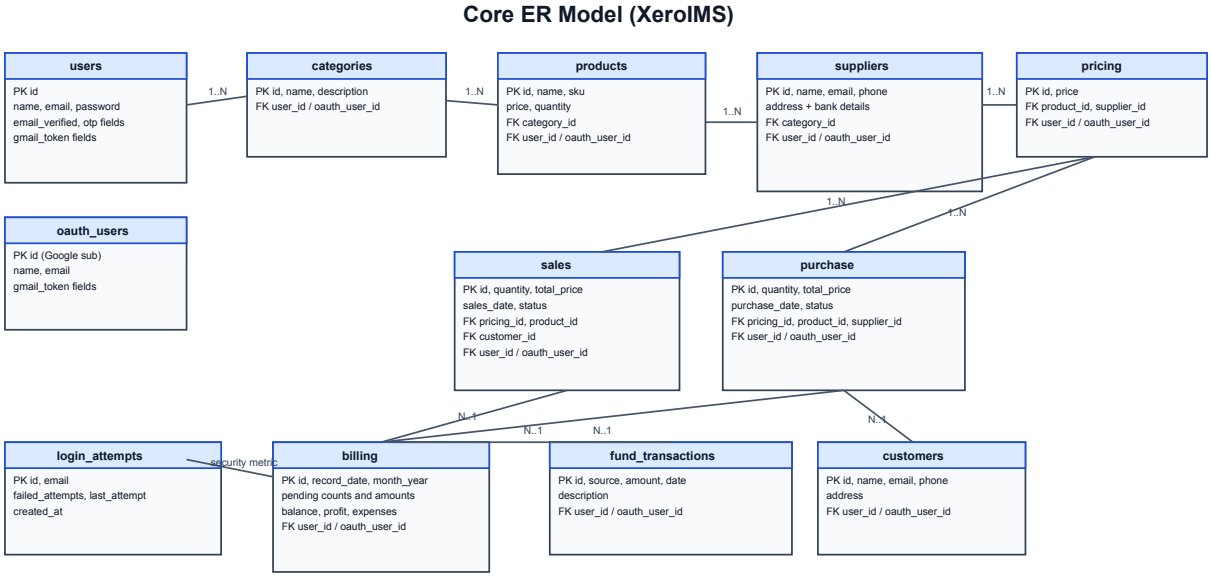


Figure 4.3: Core ER model extracted from implemented entities

4.4 Authentication and Transaction Flow

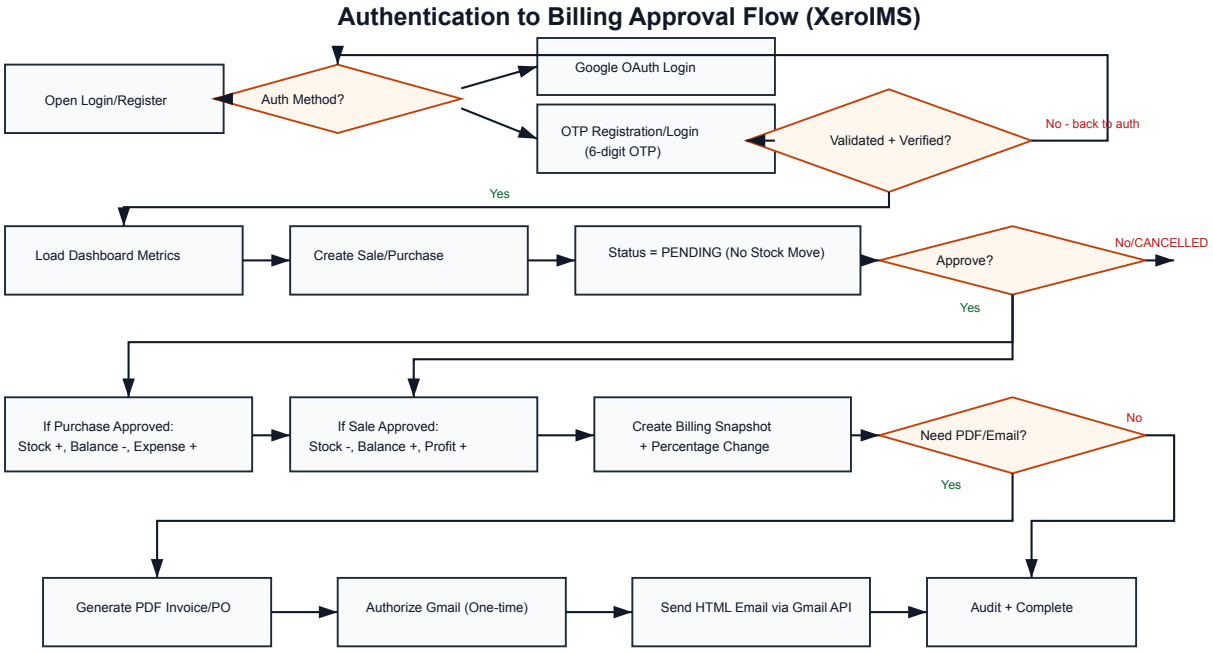


Figure 4.4: Combined authentication and billing workflow

4.5 Data Flow Diagrams (DFD)

4.5.1 DFD Level 1

DFD Level 1 expands the major system functions into operational processes:

- User authentication and verification,
- Master data management (category, product, supplier, pricing),
- Transaction processing (sale/purchase),
- Approval and billing update process,
- PDF and email communication process.

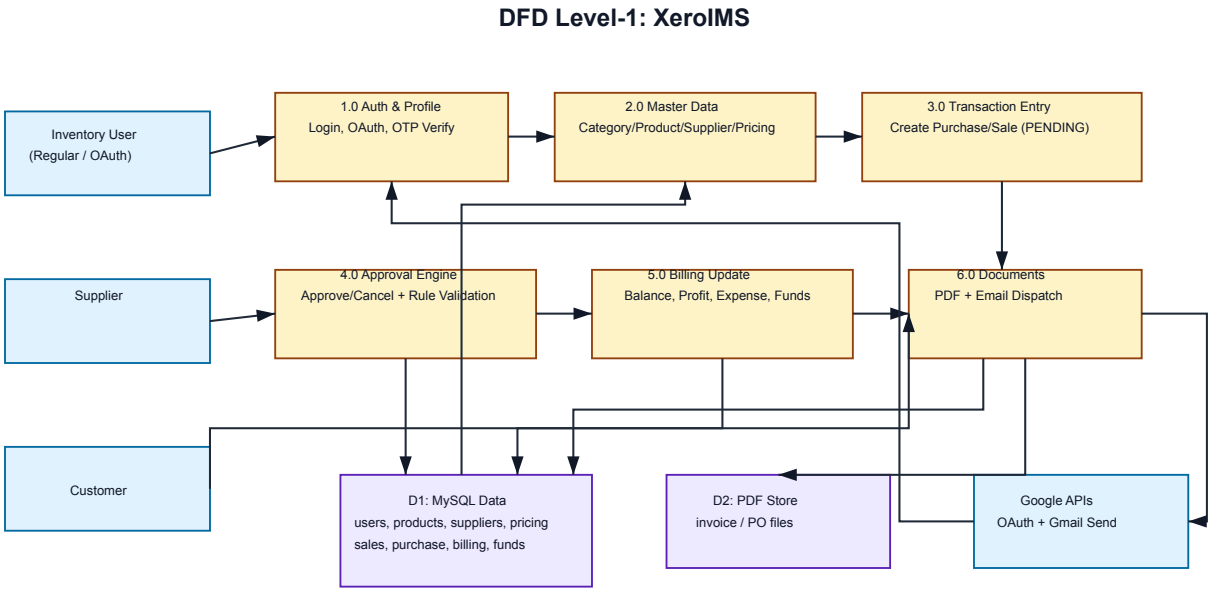


Figure 4.5: DFD Level-1 for XeroIMS

4.5.2 DFD Level 2

DFD Level 2 presents detailed sub-processes for authentication and billing workflows, including OTP validation, pending transaction approval checks, stock/fund verification, and document dispatch.

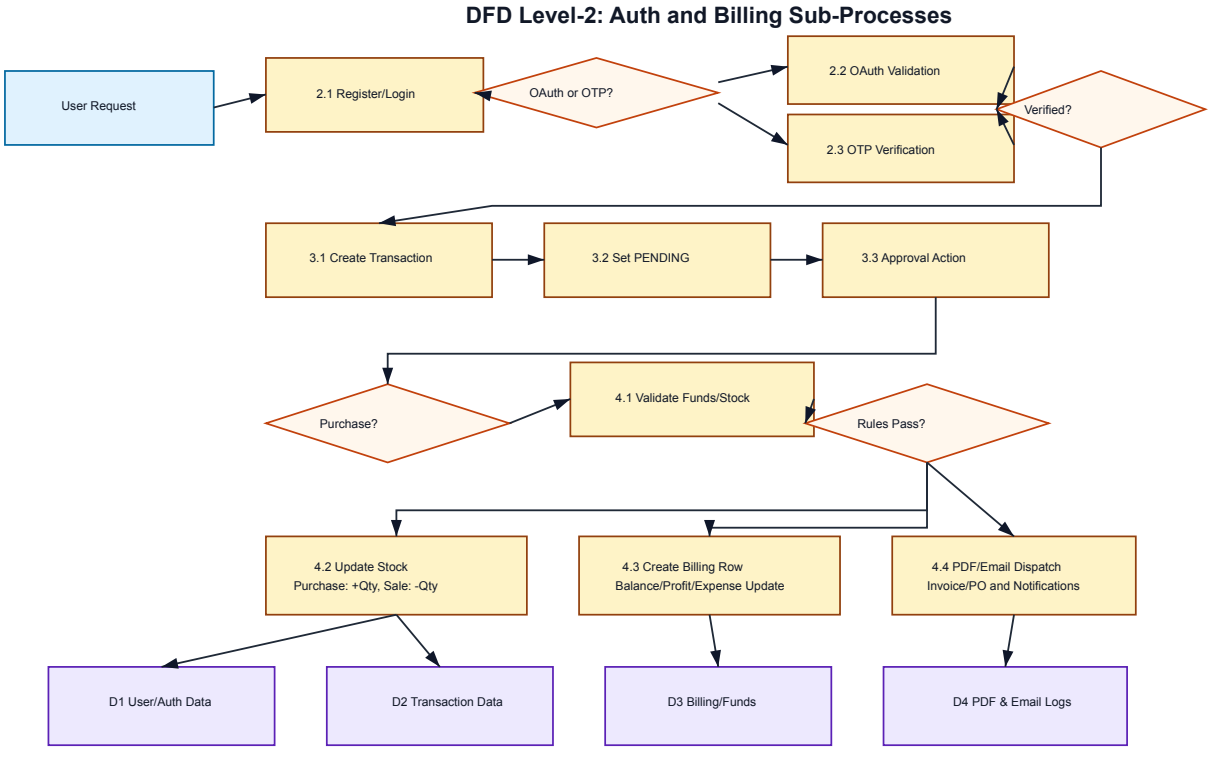


Figure 4.6: DFD Level-2 for XeroIMS

4.6 Design Decisions

4.6.1 Ownership Pattern

Instead of a dedicated tenant table, entities include nullable ownership links to either `User` or `OauthUser`. Service-level logic routes query paths accordingly.

4.6.2 Approval-Driven Inventory and Finance

Sales and purchases are first created as `PENDING`. Inventory and billing state mutate only on status transition to `COMPLETED`. This avoids premature stock and accounting changes.

4.6.3 Billing Snapshot Strategy

A new billing row is inserted for state changes rather than in-place mutation. Let:

$$B_t = \text{balance at time } t, \quad P_t = \text{profit at time } t, \quad E_t = \text{expenses at time } t$$

For completed sale amount S:

$$B_{t+1} = B_t + S, \quad P_{t+1} = P_t + S, \quad E_{t+1} = E_t$$

For completed purchase amount C:

$$B_{t+1} = B_t - C, \quad P_{t+1} = P_t - C, \quad E_{t+1} = E_t + C$$

4.6.4 Security Boundary Placement

- Nginx performs edge controls: request limiting and response headers.
- Spring Security enforces route access and login behavior.
- Account lockout information is persisted in MySQL for restart durability.

4.7 Interface Design Notes

The UI is implemented using Thymeleaf templates and static CSS/JS. Workflow-specific templates are provided for all modules (dashboard, billing, product/category/pricing forms, auth pages, and profile management).

Chapter 5

Implementation

5.1 Implementation Overview

The implementation is organized under `src/main/java/com/darkhex/xeroims` with modular packages for controller, service, model, repository, config, security, dto, and exception handling.

5.2 Package-Wise Implementation

5.2.1 Controllers

Controllers expose endpoint-driven workflows for each module:

- Authentication and account flows: registration, OTP verify/resend, forgot/reset password.
- Inventory modules: category, product, supplier, pricing.
- Transaction modules: sale, purchase, billing approvals and exports.
- PDF APIs: invoice/receipt/purchase-order generation and preview.
- Gmail OAuth callbacks and authorization status checks.

5.2.2 Services

Business logic is concentrated in service classes. Notable implementations:

- **UserService**: OTP generation, email verification, reset-password OTP flow.
- **LoginAttemptService**: lockout checks and failed attempt persistence.
- **SaleService/PurchaseService**: creation and inventory constraints.
- **BillingService**: status transitions, funds checks, metric snapshots.
- **PdfGenerationService**: generic template-driven PDF generation.
- **GmailService**: token-aware Gmail API email dispatch.

5.2.3 Data Access Layer

Repositories use derived query methods for ownership-aware retrieval and sorting by business timestamps/IDs. This keeps controller and service code concise while preserving per-user data boundaries.

5.3 Security Implementation

5.3.1 Spring Security Configuration

- Public endpoints include login/register/verify/reset routes and static assets.
- Gmail OAuth API endpoints require authenticated users.
- Form-login uses email as username and custom success/failure handlers.
- OAuth login integrates custom OAuth2 user service and dashboard redirect.

5.3.2 Account Lockout

Failed login attempts are stored in `login_attempts`. Policy implemented in service:

- Max attempts: 5
- Lockout duration: 24 hours
- Attempts reset on successful login

5.4 PDF and Email Implementation

5.4.1 PDF Pipeline

1. Map sale/purchase entities to invoice DTOs,
2. Render Thymeleaf HTML templates,
3. Convert HTML to PDF via Flying Saucer,
4. Return binary response as attachment/preview.

5.4.2 Gmail API Pipeline

1. User authorizes Gmail via OAuth callback endpoint,
2. Access/refresh tokens stored in user or oauth user record,
3. Token refresh attempted when expiry is reached,
4. MIME message converted to Gmail API message and sent.

5.5 Deployment Implementation

5.5.1 Container Topology

- **app**: Spring Boot service,
- **mysql**: isolated database (no default public port),
- **nginx**: reverse proxy and edge controls,
- **certbot**: certificate renewal loop.

5.5.2 Configuration Management

Environment variables are used for secrets and runtime tuning (database, Google OAuth, Gmail API, SMTP, base URL, profile mode). This enables local and production portability.

5.6 Implementation Traceability Matrix

Table 5.1: Requirement-to-module traceability

Req ID	**Feature**	**Primary Modules**
FR-01..05	Authentication and account recovery	SecurityConfig, RegisterController, EmailVerificationController, ForgotPasswordController, UserService, LoginAttemptService
FR-06..09	Inventory master modules	CategoryController/Service, ProductController/Service, SupplierController/Service, PricingController/Service
FR-10..16	Transaction and billing approval model	SaleController/Service, PurchaseController/Service, BillingController/Service
FR-17..19	PDF + email communication	PdfController, PdfGenerationService, EmailTemplateService, GmailOAuthController, GmailService
FR-20..21	Dashboard and exports	DashboardController/Service, BillingController (Excel export)

Chapter 6

Results and Evaluation

6.1 Validation Approach

Evaluation is based on implemented code paths, endpoint behavior, templates, and deployment configuration. Since the current automated test set is minimal (context-load test), this section emphasizes functional verification and architectural readiness.

6.2 Observed Functional Outcomes

Table 6.1: Feature validation summary from implementation artifacts

Capability	**Status**	**Evidence in Codebase**
Google OAuth login	Implemented	Spring OAuth client config + custom OAuth user service + security OAuth login flow
OTP registration/verification	Implemented	User service OTP generation and verification controller endpoints
Forgot/reset password via OTP	Implemented	ForgotPasswordController + reset logic in UserService
Login lockout persistence	Implemented	LoginAttempt model/repository/service with threshold and unlock period
Inventory master CRUD	Implemented	Category/Product/Supplier/Pricing controllers + services
Purchase/Sale pending approval	Implemented	BillingService state transition methods and controller actions
Billing and fund tracking	Implemented	Billing + FundTransaction models and metrics in billing service/controller
PDF generation	Implemented	PdfGenerationService + PdfController + PDF templates
Gmail API send	Implemented	Gmail OAuth controller + Gmail service send pipeline
Excel export	Implemented	BillingController export-sales and export-purchases endpoints
Docker + Nginx deployment	Implemented	Docker Compose, Nginx config, SSL helper scripts

6.3 Security and Reliability Observations

- Multi-layer hardening exists through Nginx edge controls and app-layer security configuration.
- Account lockout state is durable across container restarts as it is persisted in MySQL.
- Sensitive configuration is externalized to environment variables.
- MySQL is not exposed by default in compose, reducing attack surface.

6.4 Performance and Scalability Notes

XeroIMS is suitable for small to medium workloads where transactional consistency and operational clarity are prioritized. The deployment model allows future scale-outs by introducing:

- read/write query optimization,
- caching layers for dashboard aggregates,
- asynchronous email/document queues,
- horizontally scaled application instances behind Nginx.

6.5 Known Gaps

- Automated testing is minimal; module-level and integration tests should be expanded.
- Production cookie secure flag and HTTPS assumptions should be reviewed and enforced in final deployment profile.
- Some controllers process large in-memory filtering for exports; pagination-aware DB filtering can improve scale.

6.6 Outcome Summary

Overall, XeroIMS demonstrates a complete and practical implementation of a secure inventory management workflow with authentication alternatives, approval-driven accounting, and integrated business communication/document tools.

6.7 Result Screenshots

6.7.1 Authentication and Account Screens

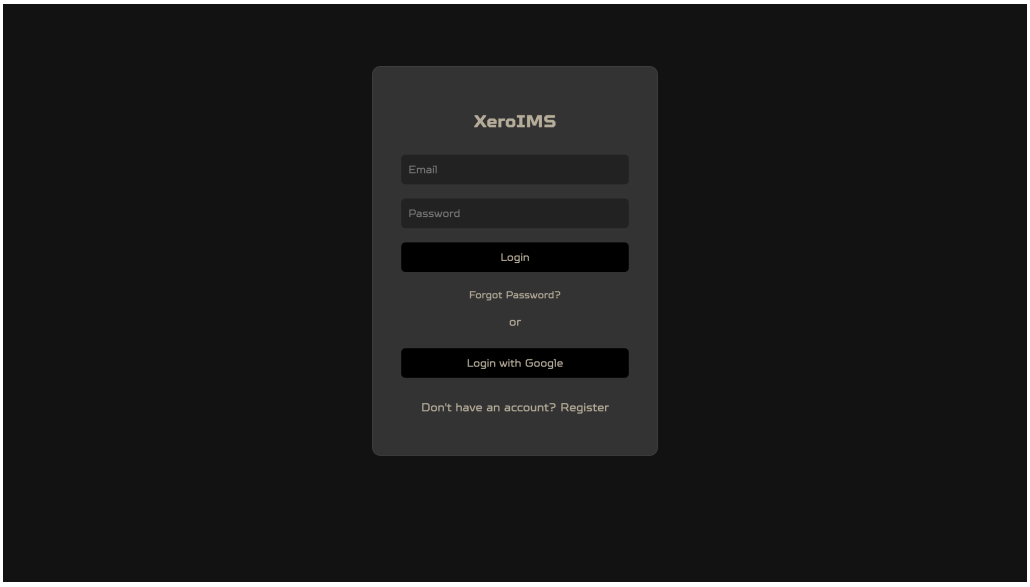


Figure 6.1: Login page

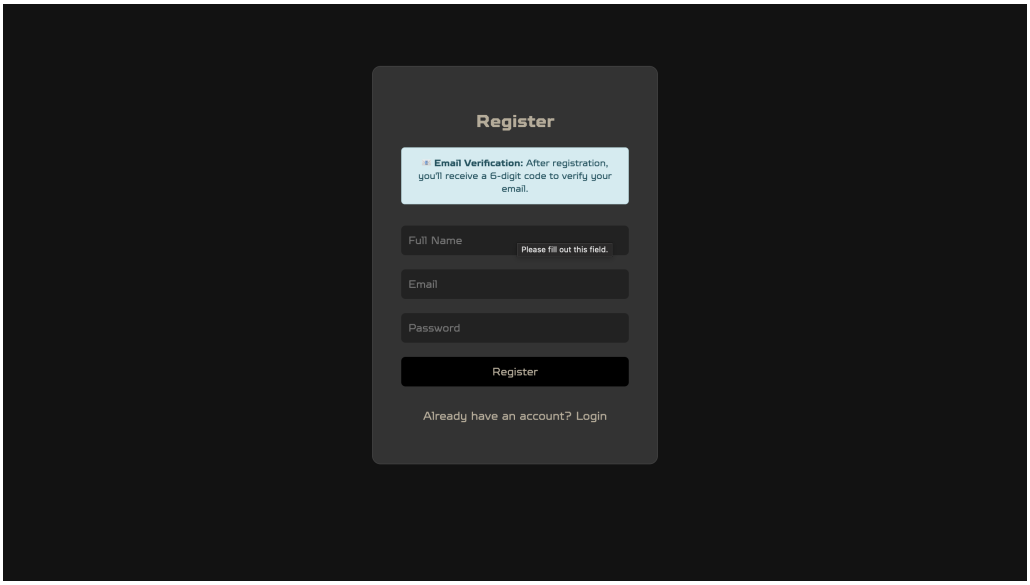


Figure 6.2: Registration page

6.7.2 Dashboard Screens

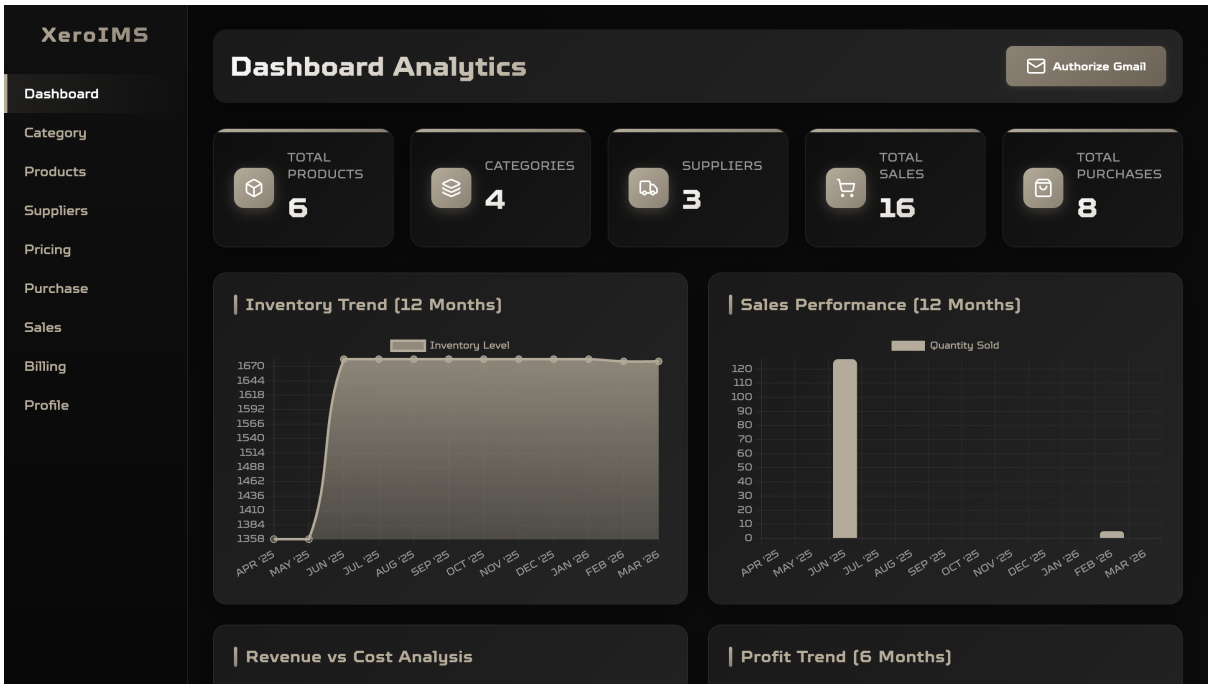


Figure 6.3: Dashboard view I

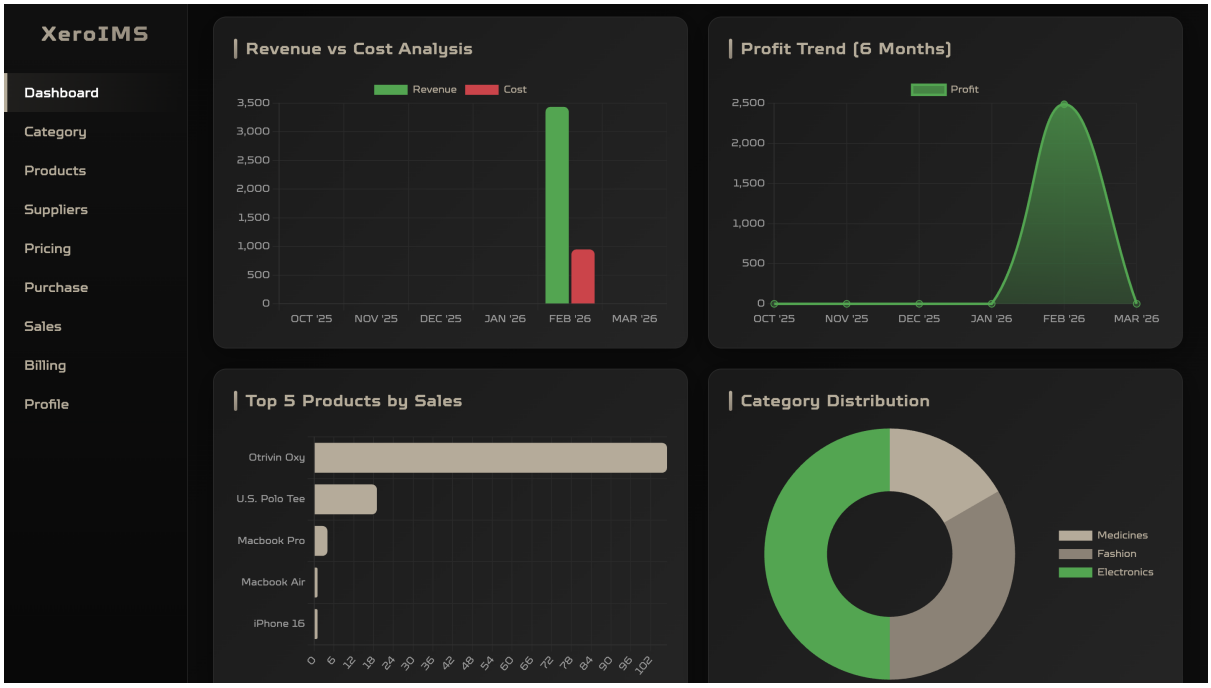


Figure 6.4: Dashboard view II

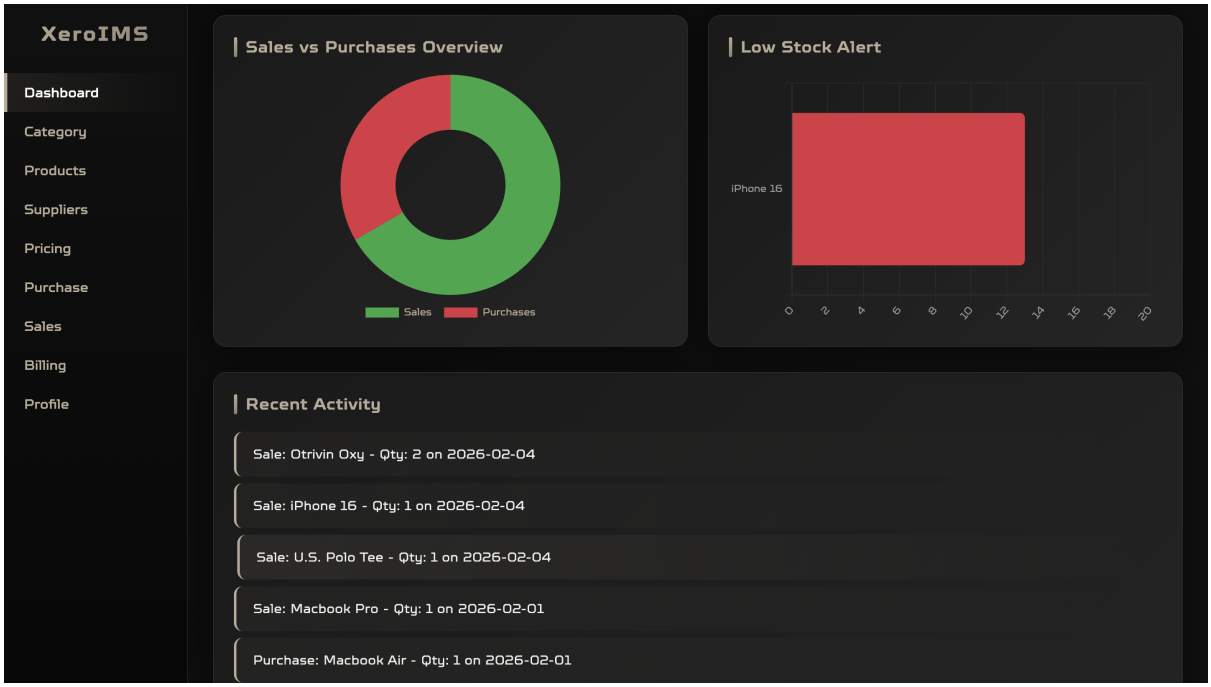


Figure 6.5: Dashboard view III

6.7.3 Core Module Screens

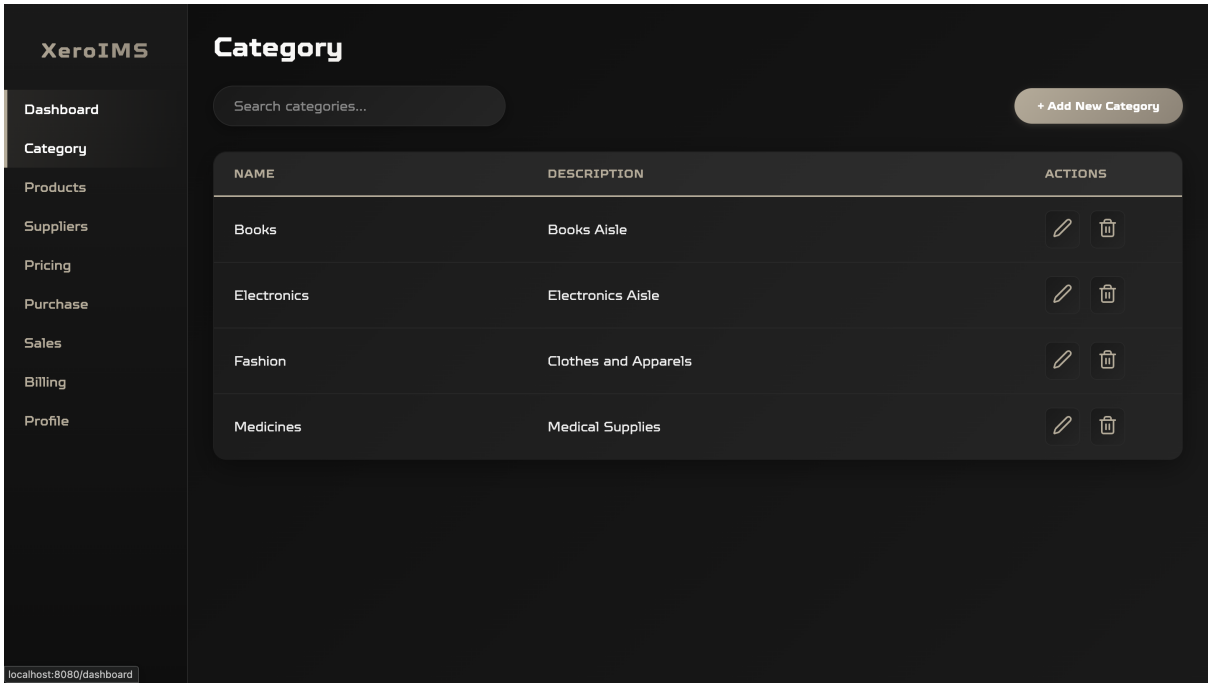


Figure 6.6: Category management module

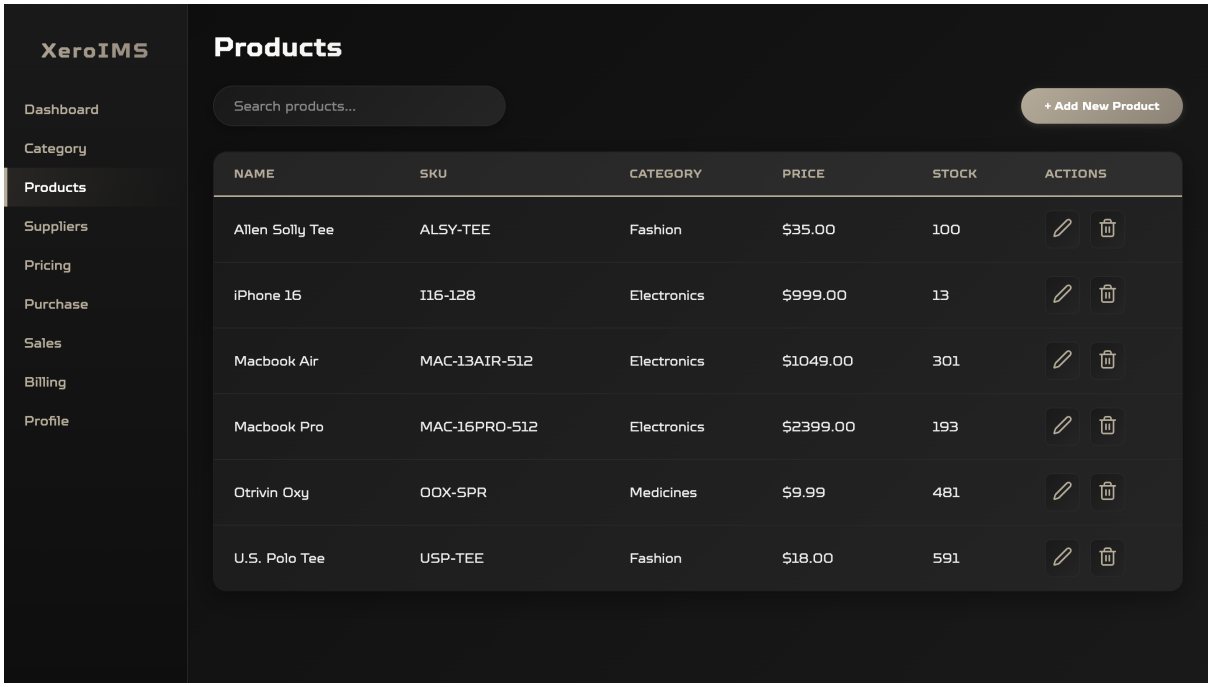


Figure 6.7: Product management module

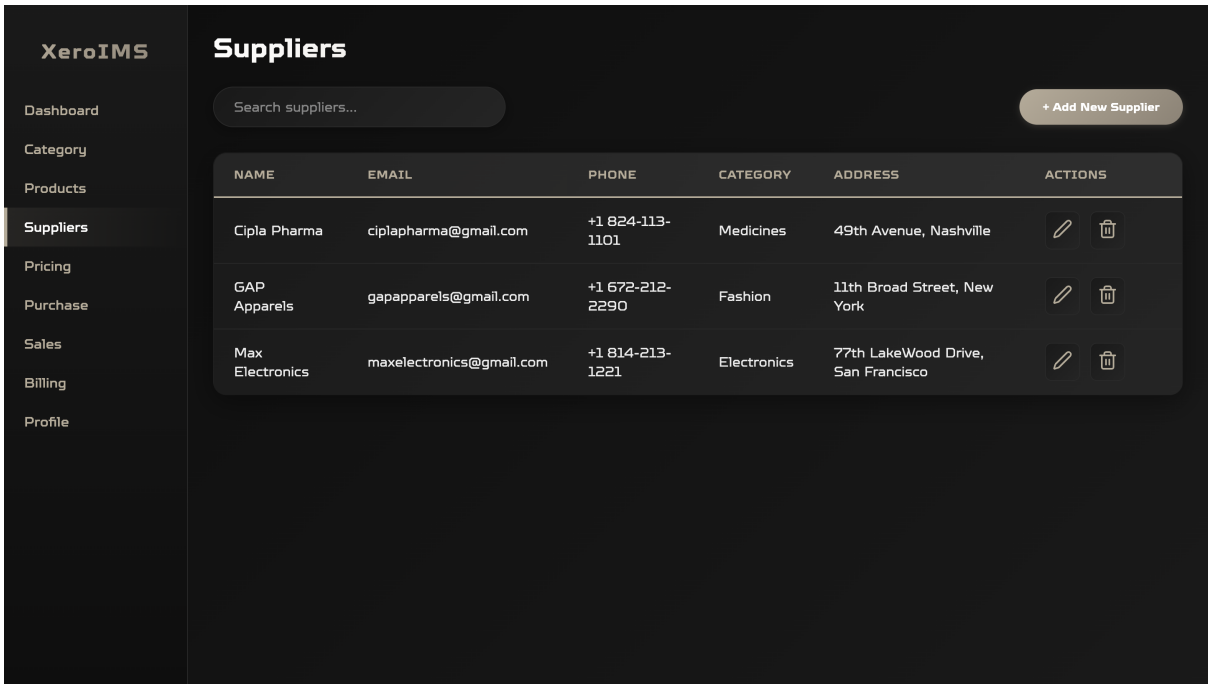


Figure 6.8: Supplier management module

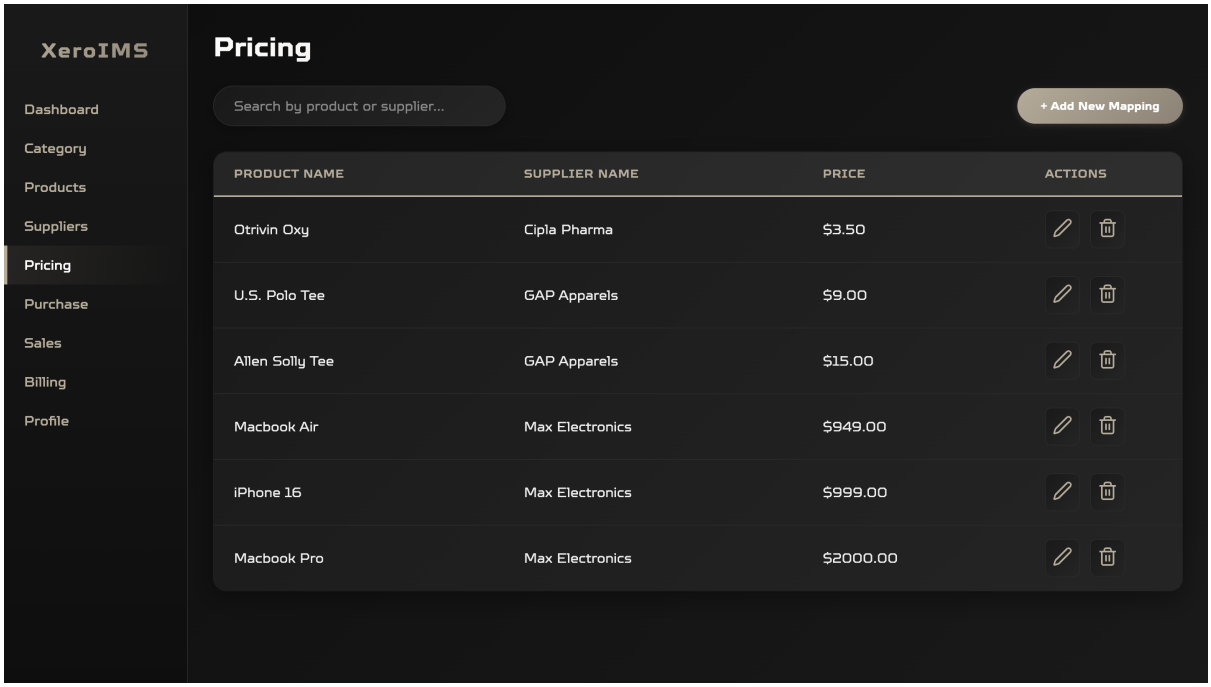


Figure 6.9: Pricing management module

6.7.4 Transaction and Billing Screens

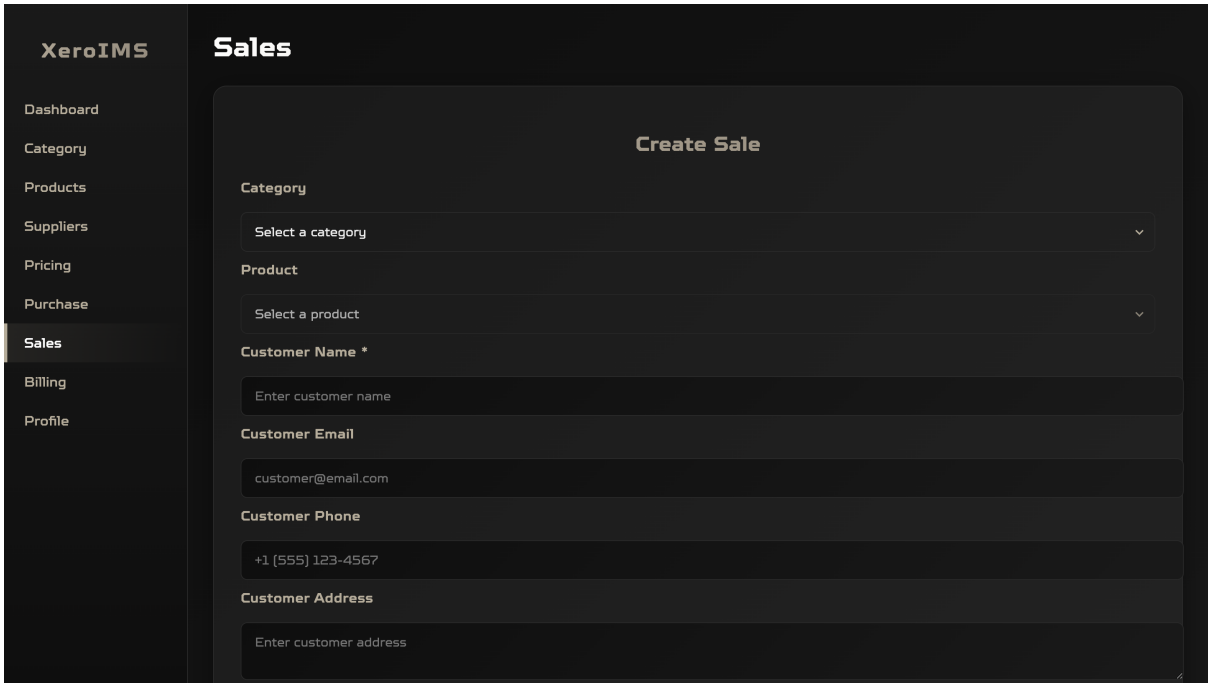


Figure 6.10: Sales module

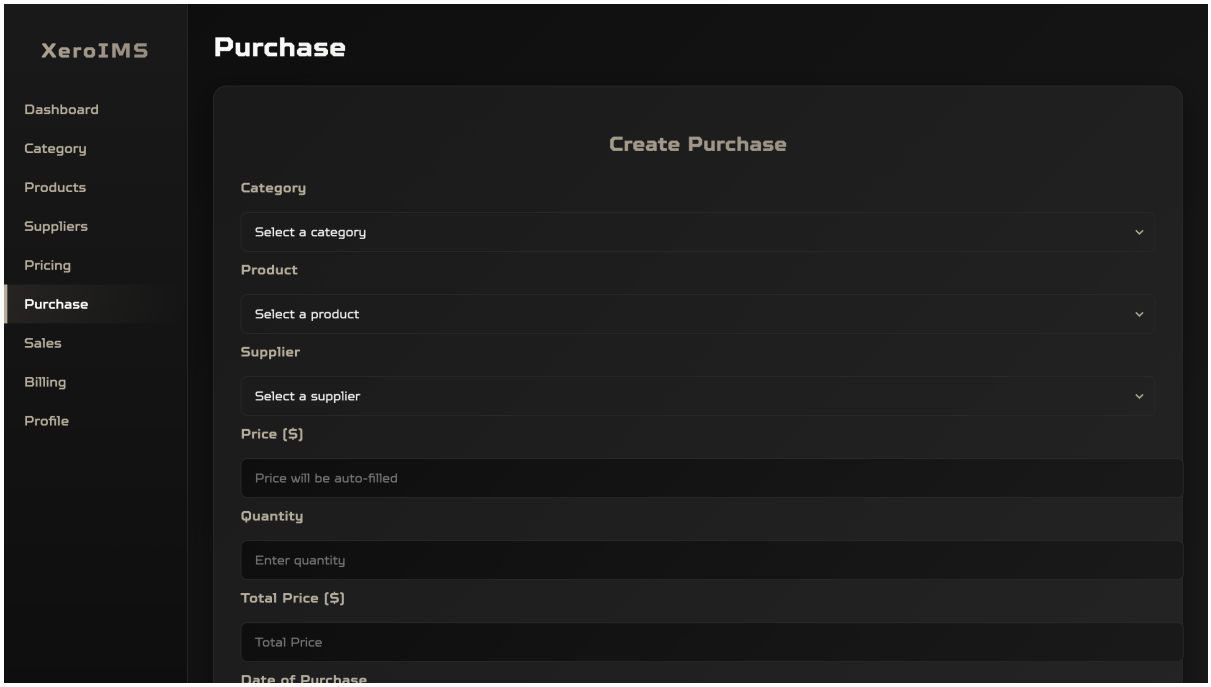


Figure 6.11: Purchase module

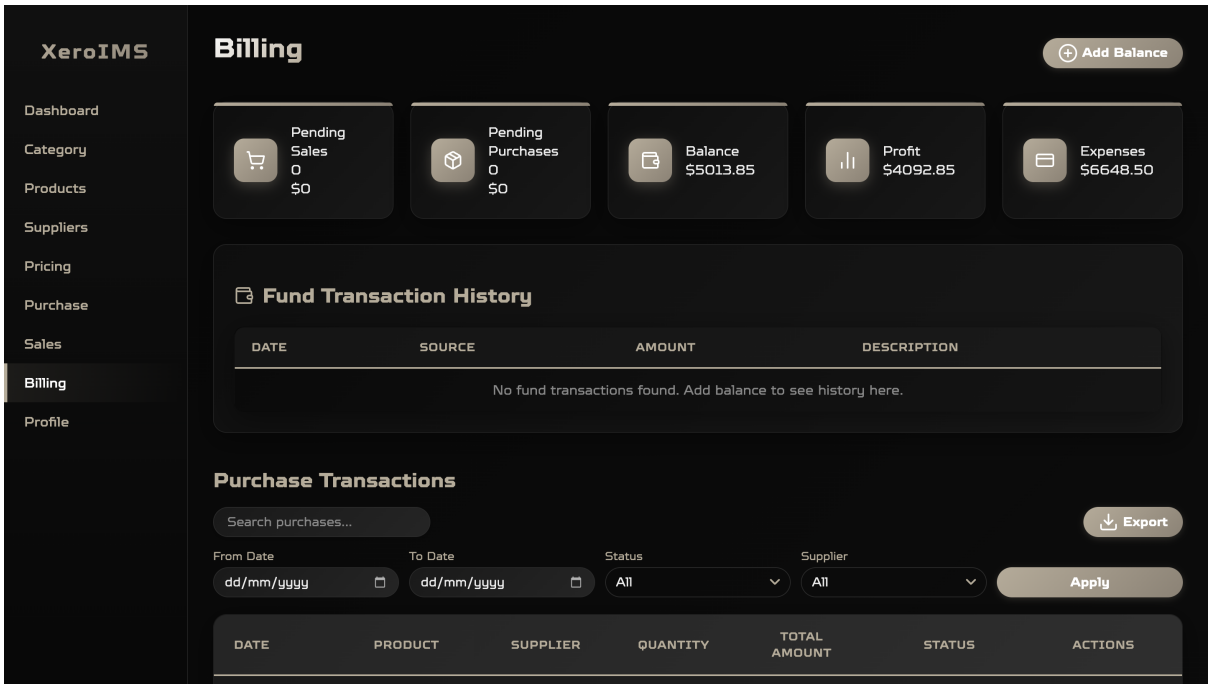


Figure 6.12: Billing module view I

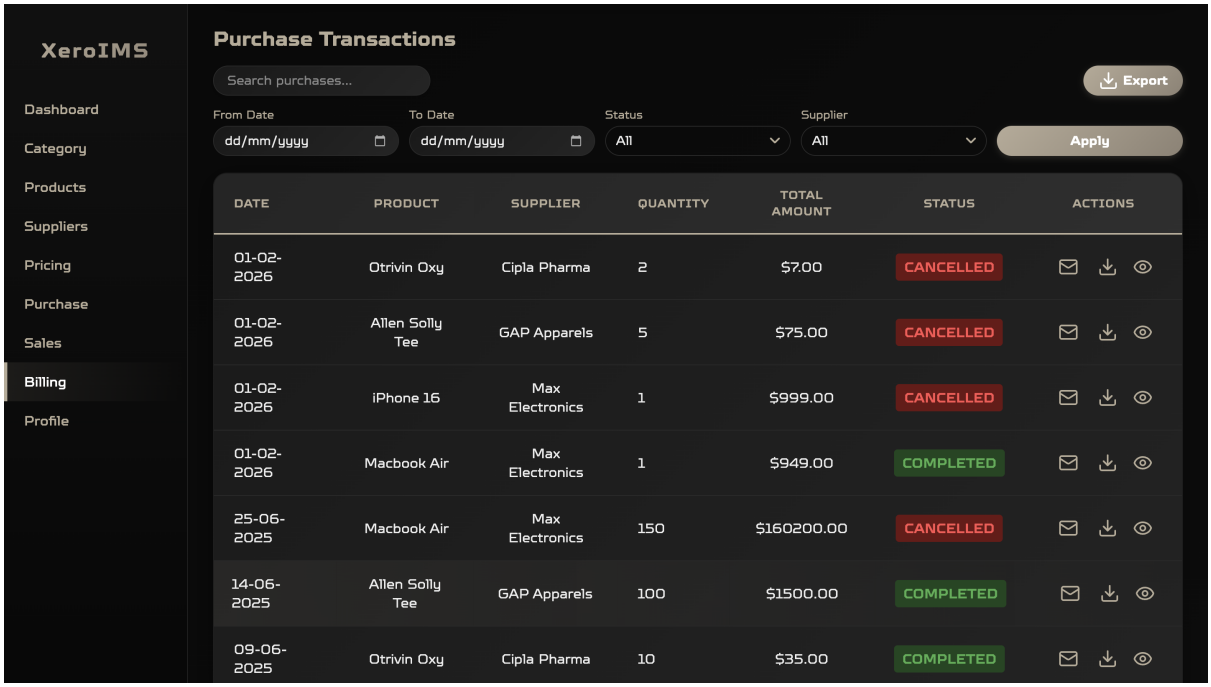


Figure 6.13: Billing module view II

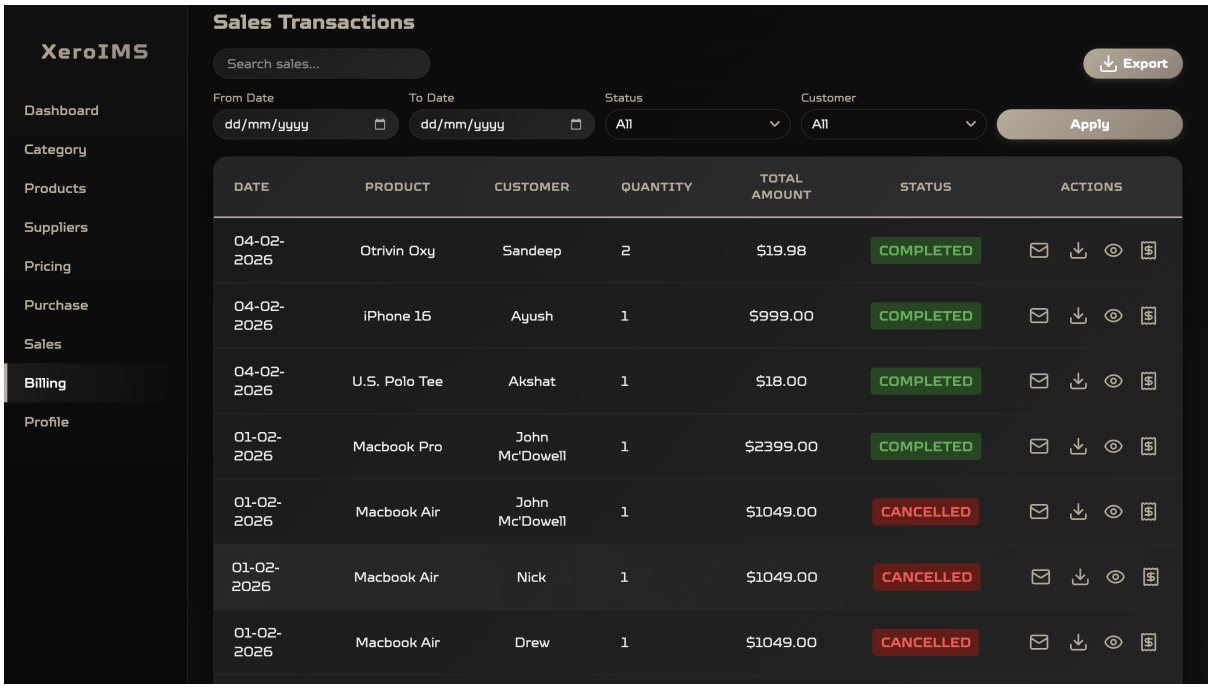


Figure 6.14: Billing module view III

6.7.5 Profile Screens

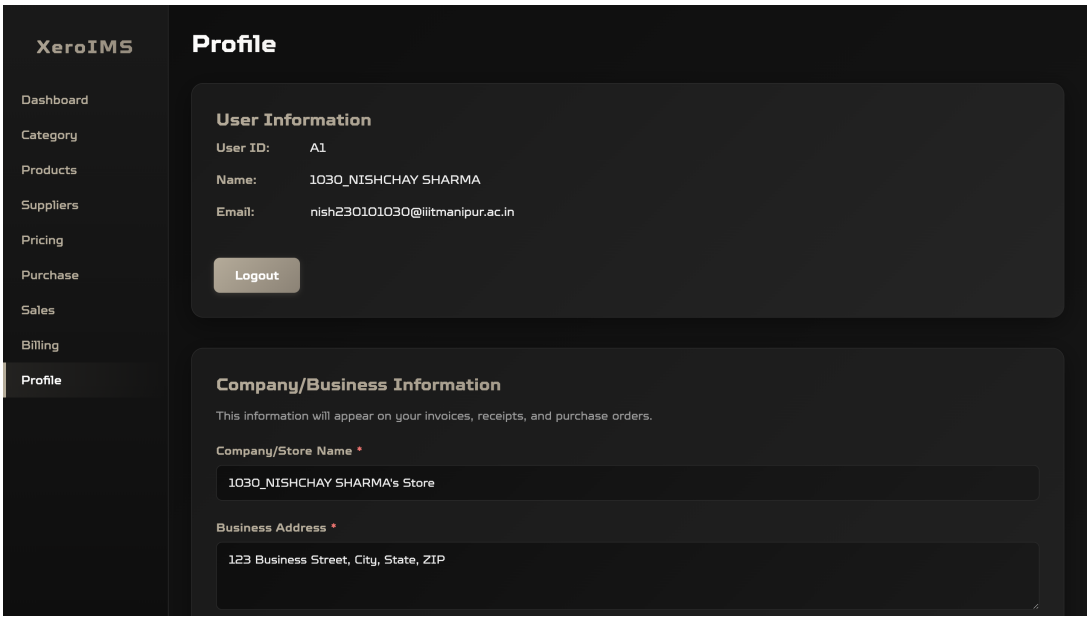


Figure 6.15: Profile page view I

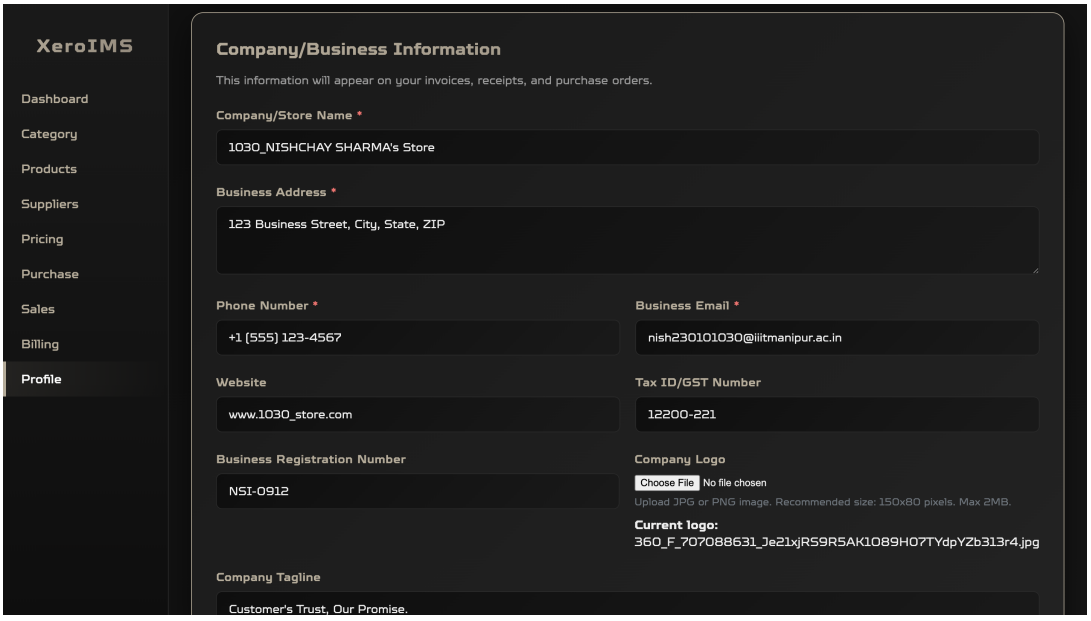


Figure 6.16: Profile page view II

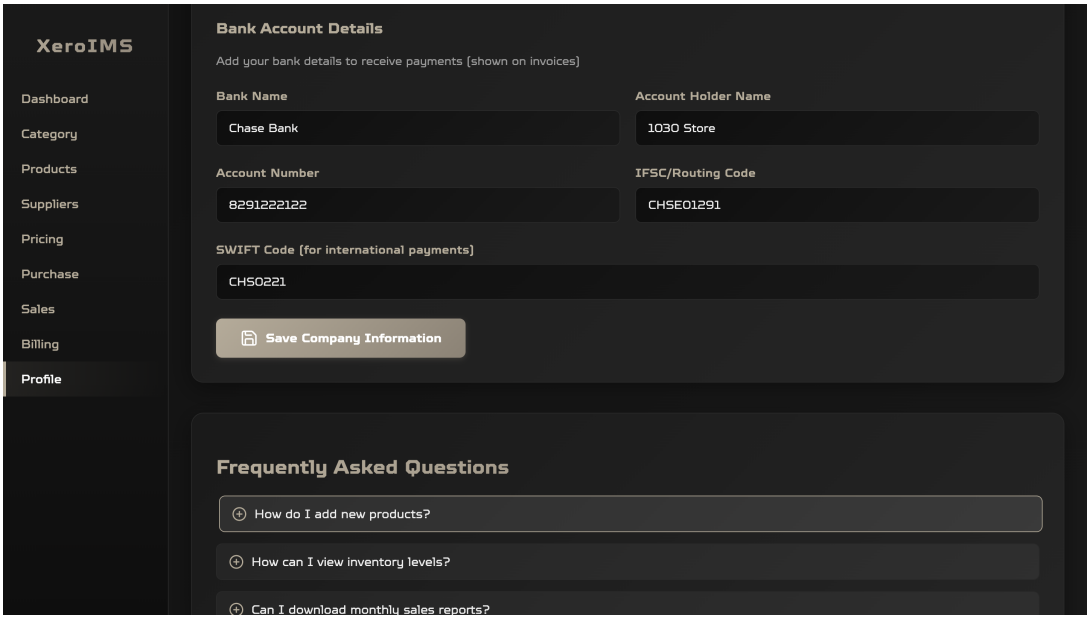


Figure 6.17: Profile page view III

6.8 Generated PDF Documents and Delivery Mode

The generated billing documents validate implemented behavior:

- **Receipt** is generated as a physical print copy accompanying product delivery.
- **Sale Order PDF** is dispatched through email.
- **Purchase Order PDF** is dispatched through email.



1030_NISHCHAY SHARMA's Store

123 Business Street, City, State, ZIP
+1 (555) 123-4567
nish230101030@iiitmanipur.ac.in

RECEIPT

STATUS: COMPLETED

Receipt #: INV-000001
Date: 05/02/2026
Customer: Akshat

Item **Qty** **Price**

U.S. Polo Tee
2 $39.98
USPA-TEE @$19.99

Subtotal: $39.98

TOTAL: $39.98

Paid: $39.98

Thank you for your business!

*** CUSTOMER COPY ***

Figure 6.18: Generated receipt (physical copy with product)



INVOICE

INV-000030

COMPLETED

1030_NISHCHAY SHARMA's Store

Customer's Trust, Our Promise.

123 Business Street, City, State, ZIP

+1 (555) 123-4567

nish230101030@iiitmanipur.ac.in

www.1030_store.com

Tax ID: 12200-221

Bill To

John Mc'Dowell

john@gmail.com

NA

NA

Invoice Details

Invoice Date:

01/02/2026

Due Date:

03/03/2026

#	DESCRIPTION	QUANTITY	UNIT PRICE	TOTAL
1	**Macbook Pro** SKU: MAC-16PRO-512	1	$2399.00	$2399.00

Subtotal:	**$2399.00**
TOTAL:	**$2399.00**
Paid Amount:	**$2399.00**

Figure 6.19: Generated sale-order PDF (emailed)



PURCHASE ORDER

PO-000073

COMPLETED

1030_NISHCHAY SHARMA's Store

Customer's Trust, Our Promise.

123 Business Street, City, State, ZIP

+1 (555) 123-4567

nish230101030@iiitmanipur.ac.in

www.1030_store.com

Tax ID: 12200-221

Vendor/Supplier

Max Electronics

maxelectronics@gmail.com

+1 814-213-1221

77th LakeWood Drive, San Francisco

Order Details

Order Date:

01/02/2026

Expected Delivery:

08/02/2026

#	DESCRIPTION	QUANTITY	UNIT PRICE	TOTAL
1	**Macbook Air** SKU: MAC-13AIR-512	1	$949.00	$949.00

Subtotal:	**$949.00**
TOTAL:	**$949.00**

SUPPLIER PAYMENT INFORMATION

Figure 6.20: Generated purchase-order PDF (emailed)

Chapter 7

Conclusion and Future Scope

7.1 Conclusion

XeroIMS achieves the core goals of a modern, self-hosted inventory management system by integrating secure authentication, ownership-aware data handling, transaction approval controls, financial tracking, and automated document/email outputs. The architecture is modular and deployable, balancing development simplicity with production-minded controls.

The project demonstrates that the selected stack (Spring Boot + MySQL + Docker + Nginx) can deliver business-grade features typically expected in inventory software.

7.2 Contributions of XeroIMS

- Combined Google OAuth and OTP-based account lifecycle.
- Approval-centric purchase/sales model tied directly to stock and billing state.
- Integrated PDF generation and Gmail API communication flow.
- Deployment blueprint with reverse proxy security, health checks, and persistent storage.

7.3 Future Scope

1. Add comprehensive unit, integration, and end-to-end automated tests.
2. Introduce role-based access control (owner/operator/accountant/auditor).
3. Add asynchronous job queues for PDF/email generation at scale.
4. Build richer audit trails and observability dashboards.
5. Support multi-warehouse inventory and batch-level tracking.
6. Add REST API versioning and optional SPA/mobile clients.

7.4 Final Remark

XeroIMS provides a strong technical base for academic demonstration and real-world extension. With testing expansion and operational hardening, it can evolve into a production-grade inventory platform for broader organizational use.

Bibliography

[1] VMware, Inc. Spring boot reference documentation. https://docs.spring.io/spring-boot/docs/current/reference/html/, 2026.

[2] D. Hardt. The oauth 2.0 authorization framework (rfc 6749). https://www.rfc-editor.org/rfc/rfc6749, 2012.

[3] Flying Saucer Project. Flying saucer project documentation. https://github.com/flyingsaucerproject/flyingsaucer, 2026.